

# User Guide

WebFlowMaster Enterprise Documentation

Language: IT | Version 1.0.0

Generated on: 5/5/2026





# WebFlowMaster: Guida Utente Avanzata

Strategie e Operazioni per il Quality Assurance Moderno



## Visione d'Insieme

WebFlowMaster non è solo un tool di automazione, ma un ecosistema completo per il **Continuous Testing**. Questa guida ti accompagnerà dalla creazione del tuo primo test fino alla gestione di suite di regressione complesse in ambienti enterprise.



## Sommario

1. [Fondamenti del Test Automation](#)
2. [Mastering the Visual Builder](#)
  - [Gestione dei Selettori \(CSS & XPath\)](#)
  - [Variabili e Data Injection](#)
  - [Logiche di Attesa \(Wait Strategies\)](#)
3. [Registrazione Avanzata con il Live Engine](#)
4. [API Validation & Integration Testing](#)
5. [Analisi Strategica dei Risultati](#)
  - [Visual Regression Check](#)
  - [Debugging tramite Log di Rete](#)
6. [Best Practices per Test Robusti](#)

## 1. Fondamenti del Test Automation

In WebFlowMaster, ogni test è composto da una **Test Sequence**. Una sequenza è una catena di azioni atomiche eseguite in un browser headless o headful. **Punto Chiave:** Un test efficace deve essere *deterministico*. Lo stesso test, eseguito 100 volte senza modifiche al codice, deve produrre lo stesso risultato.

## 2. Mastering the Visual Builder

Il Visual Builder è il cuore della piattaforma. Ti permette di visualizzare il flusso logico del test come un grafo direzionale.

### A. Gestione dei Selettori (CSS & XPath)

WebFlowMaster utilizza algoritmi avanzati per identificare gli elementi. Tuttavia, per test enterprise robusti, segui queste regole:

- **ID Statici:** Usa sempre ID se presenti ( `#submit-button` ).
- **Data Attributes:** La best practice è usare attributi dedicati al test, come `[data-testid="login-btn"]` .
- **XPath Dinamici:** Usa XPath solo quando devi navigare la gerarchia DOM in modo relativo (es. "trova il pulsante accanto al testo 'Email'").

## B. Variabili e Data Injection

Puoi rendere i tuoi test dinamici utilizzando le variabili:

- `{{ENV_URL}}` : URL dell'ambiente selezionato.
- `{{RANDOM_EMAIL}}` : Genera automaticamente una mail univoca per test di registrazione.
- **Custom Parameters:** Passa parametri personalizzati durante il lancio del test per testare scenari diversi con lo stesso flusso.

## C. Logiche di Attesa (Wait Strategies)

L'errore più comune nei test è la mancanza di sincronizzazione. WebFlowMaster offre tre tipi di attesa:

1. **Implicit Wait:** Il sistema aspetta che l'elemento sia presente nel DOM.
2. **Explicit Wait:** Forza un'attesa di X millisecondi (usa con cautela).
3. **Condition Wait:** Aspetta che una specifica condizione sia vera (es. un caricamento a schermo che scompaia).

---

## 3. Registrazione Avanzata con il Live Engine

---

Il motore di registrazione non si limita a catturare i click. Cattura l'intento.

- **Hover Actions:** Muovi il mouse lentamente per catturare menu a comparsa.
- **Keyboard Events:** Supporto completo per Tab, Enter e combinazioni di tasti.
- **Smart Detection:** Durante la registrazione, il sistema suggerisce automaticamente asserzioni basate sull'elemento cliccato.

---

## 4. API Validation & Integration Testing

---

Il modulo API ti permette di testare il "cuore" delle tue applicazioni.

- **Chaining:** Usa il risultato di una chiamata API (es. un Token di autenticazione) come input per lo step successivo.
- **Schema Validation:** Carica uno schema JSON per validare istantaneamente che la risposta del server sia strutturalmente corretta.

- **Status Assertions:** Definisci range di codici HTTP accettabili (es. 200-204).
- 

## 5. Analisi Strategica dei Risultati

---

### A. Visual Regression Check

WebFlowMaster confronta l'interfaccia attuale con una "Baseline". Se una modifica al CSS sposta un pulsante anche solo di 5 pixel, il test segnalerà un **Warning**. Questo è fondamentale per prevenire regressioni estetiche inaspettate.

### B. Debugging tramite Log di Rete

Ogni report include il tab "**Network Trace**". Se un test fallisce, puoi vedere se una chiamata API sottostante è andata in timeout o ha restituito un errore 500, identificando immediatamente se il problema è nel frontend o nel backend.

---

## 6. Best Practices per Test Robusti

---

1. **Atomicità:** Ogni test dovrebbe verificare una sola funzionalità.
  2. **Indipendenza:** Un test non dovrebbe dipendere dal successo di un test precedente.
  3. **Cleanup:** Assicurati che il test pulisca i dati creati (o usi ambienti isolati).
  4. **No Flakiness:** Se un test fallisce in modo intermittente, analizza i tempi di caricamento e ottimizza le strategie di attesa.
- 

© 2026 WebFlowMaster Enterprise. Tutti i diritti riservati.